

مشروع: هندسة البيانات اللحظية من عدادات الكهرباء الذكية
Real-Time Data Engineering for Smart Electricity Meters

Done By

Eng. Ahmed Al-Aini

Supervised By

Eng. Ahmed El-Saeed

1- Project Idea & Business Scenario

This project focuses on building a real-time data pipeline to simulate an electricity consumption monitoring system based on Smart Meters.

The scenario assumes a network of smart meters installed across various buildings, continuously transmitting periodic readings. These readings include critical operational data such as:

- Consumption Rate (Power)
- Voltage Levels (Voltage)
- Current Value (Current)
- Meter Status (Active, Inactive, Fault)
- Timestamp for each reading.

2. Data Generation & Simulation

To represent this scenario realistically, a Python-based data generator was developed to simulate a continuous stream of meter data by generating sequential CSV files.

The generated data includes both valid readings and intentional anomalies to test the system's resilience. These anomalies include:

- Missing values in certain fields.
- Data type inconsistencies (e.g., text in numeric fields).

- Multiple timestamp formats.
- Incomplete rows (corrupted records).
- Inconsistent electrical readings (e.g., negative power).

Below is the python Script:

```
import csv
import os
import random
import time
from datetime import datetime, timedelta

OUTPUT_DIR = "./stream_data"

if not os.path.exists(OUTPUT_DIR):
    os.makedirs(OUTPUT_DIR)

def random_timestamp():
    formats = [
        "%Y-%m-%d %H:%M:%S",
        "%d/%m/%Y %H:%M",
        "%m-%d-%Y %H:%M",
    ]
    dt = datetime.now() - timedelta(seconds=random.randint(0, 10000))
    return dt.strftime(random.choice(formats))

def corrupt_row():
    return ["CORRUPT", "", "NOT_A_NUMBER", "9999", "NAN", "UNKNOWN",
"NOT_A_DATE"]

def generate_record():
    meter_id = f"MTR{random.randint(1000, 1010)}"
    building_id = f"BLD{random.randint(200, 210)}"
    power = round(random.uniform(0.5, 20.0), 2)
    voltage = random.choice([220, 230, 110, None])
    current = round(random.uniform(1, 50), 2)
```

```

status = random.choice(["ACTIVE", "INACTIVE", "FAULT"])
timestamp = random_timestamp()

# inject anomalies
if random.random() < 0.1:
    return corrupt_row()

if random.random() < 0.1:
    building_id = ""

if random.random() < 0.1:
    power = -abs(power)

if random.random() < 0.1:
    timestamp = "INVALID_DATE"
if random.random() < 0.05:
    power = "NAN" # String instead of number
if random.random() < 0.05:
    current = "INF" # Invalid numeric string
if random.random() < 0.05:
    voltage = 9999 # Unrealistic value

return [meter_id, building_id, power, voltage, current, status,
timestamp]

def get_filename():
    # filename based on current date & time
    return datetime.now().strftime("meter_data_%Y-%m-%d_%H-%M-%S.csv")

def write_file():
    filename = os.path.join(OUTPUT_DIR, get_filename())

    header = ["meter_id", "building_id", "power_kw", "voltage", "current",
"status", "timestamp"]

    records = []

    for _ in range(20):
        record = generate_record()
        records.append(record)

    # duplicates simulation
    if random.random() < 0.1:

```

```

        records.append(record.copy())

    with open(filename, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(records)

    print(f"[+] Generated: {filename} ({len(records)} records)")

def stream_generator():
    while True:
        write_file()
        time.sleep(3)

if __name__ == "__main__":
    stream_generator()

```

3. Ingestion and Chunking Stage

a. Data Ingestion Strategy:

The ingestion path is fully automated, using **ListFile** and **FetchFile** processors to monitor the `/lab_data/assignment2/stream_data` directory. This dual-processor approach was chosen for its technical advantages:

- **State Tracking:** The system maintains a record of processed files to prevent duplicates.
- **Cluster Readiness:** Unlike, GetFile this approach is highly effective in distributed (cluster) environments.
- **Responsiveness:** The polling period is set to 1 second, ensuring near-instantaneous processing.

B. Apache NiFi Core Concepts Implementation

To ensure system reliability under high data loads, several core NiFi concepts were applied:

Back Pressure

Back pressure acts as a safety valve to prevent memory overflow. The limits are set as:

- **Critical Path (ListFile → FetchFile):** 1,000 records or 100 MB.
- **Other Paths:** 10,000 records or 1 GB. When a queue reaches these limits, the source processor automatically pauses until the data is processed.

Queue Management (FIFO)

The **FirstInFirstOut (FIFO)** strategy is used on the critical path to ensure meter data is processed in the correct chronological order. This is vital for maintaining accurate consumption history and preventing data overlaps.

Provenance Tracking

NiFi's Data Provenance feature is used to monitor the lifecycle of every FlowFile (from RECEIVE and FETCH to SPLIT and SEND), facilitating technical auditing and troubleshooting.

FlowFile Attributes

Attributes like `absolute.pat${h}/${filename}` are used for programmatic routing and to build dynamic HDFS storage paths later in the pipeline.

Yield & Penalization

- **Yield Duration:** Set to 1 second to reduce CPU consumption when no new data is available.
- **Penalization:** Problematic records are "penalized" for 30 seconds, allowing healthy records to continue flowing without being blocked by a single failure.

Retry Handling

A RetryFlowFile processor manages transient errors with a 3-attempt limit. Records that fail all attempts are routed to a monitoring path for manual analysis.

c. File Chunking

A SplitText processor divides incoming files into smaller chunks not exceeding 64 KB.

- **Header Preservation:** The system is configured to copy the CSV header into every resulting chunk, ensuring data remains identifiable in later stages.
- **Efficiency:** SplitText provides high performance for text-based data with minimal resource overhead.

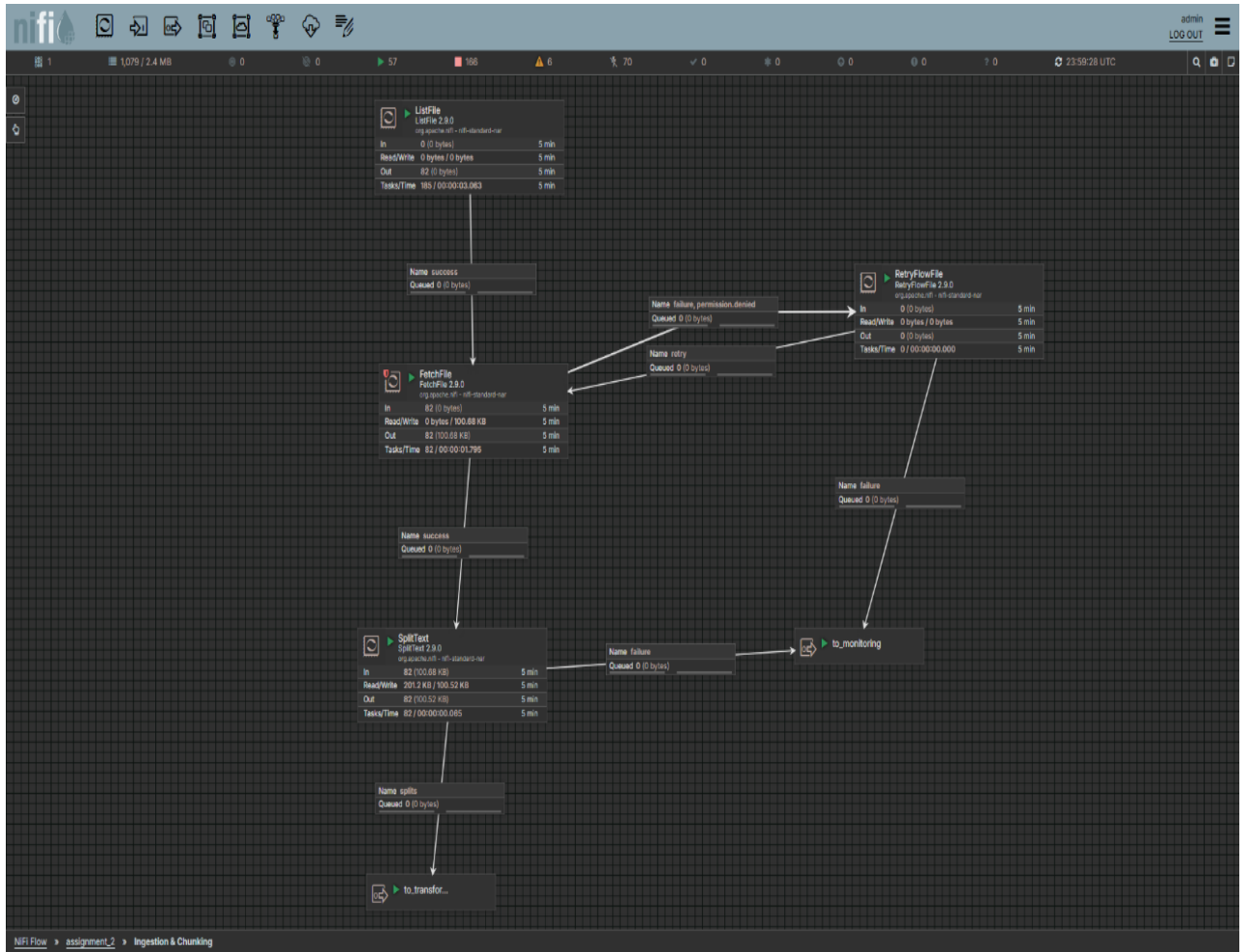


Figure 1: Ingestion and Chunking FlowFile

3. Transformation and Validation Stage

This stage acts as the "brain" of the project, validating data quality and converting it from CSV to JSON format.

a. Transformation Strategy

The transformation path uses three primary processors to ensure no corrupted data reaches the final storage:

1. **ConvertRecord**: Performs initial CSV → JSON conversion using a flexible schema.
2. **ValidateRecord**: Performs strict data type validation (e.g., ensuring readings are numbers and not "NAN").

3. **RouteOnAttribute:** Routes clean records to Kafka and suspicious records to monitoring.

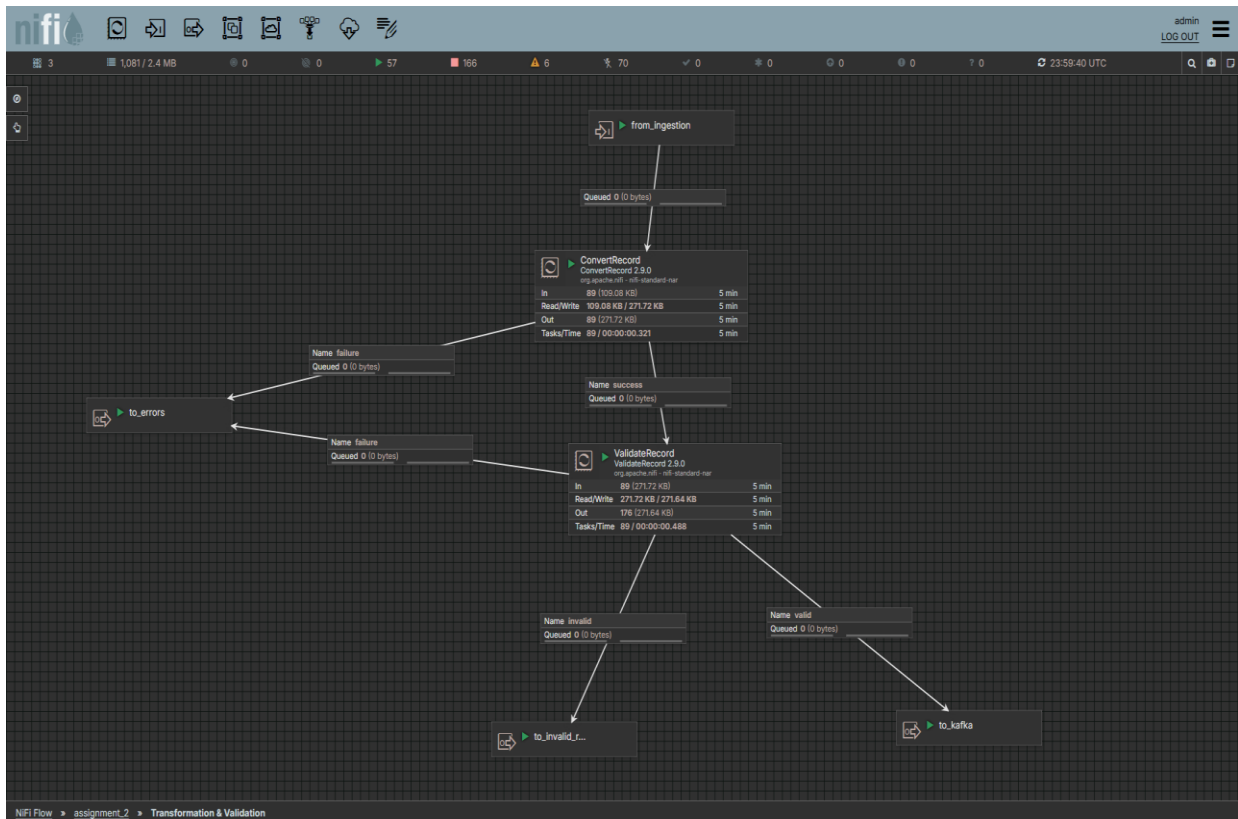


Figure 2: Transformation and Validation

b. Two-Stage Schema Strategy

To handle "noisy" data (like NAN or INF values caused by sensor faults), a two-stage schema strategy was implemented:

i. Flexible Schema (Stage 1)

The goal is to **accept all data** initially. Fields are defined as a mix of (Double, String, Null). This prevents the process from failing if a sensor sends a string like "NAN" instead of a number.

ii. Strict Schema (Stage 2)

This is the "filter" stage. The generated JSON is checked against a strict schema that only accepts real numbers (Double/Int) for readings.

- **Valid Records:** Sent to the clean path for Kafka.
- **Anomalous Records (NAN/INF):** Flagged as invalid and sent for review.

c. Data Handling Examples

Scenario	Input (CSV)	Result	Final Path
Clean Record	MTR1005,BLD205,15.5,220,8.7...	Conversion successful and schema compliant	to_kafka
Anomalous Values	MTR1006,BLD206,NAN,220,INF...	Converted successfully but failed strict validation	to_monitoring
Missing Values	MTR1007,,12.3,,8.7...	Null values are accepted as valid empty data	to_kafka
Corrupted Row	CORRUPT,,NOT_A_NUMBER,9999...	Structure failure in the row itself	to_errors

Controller Services Used

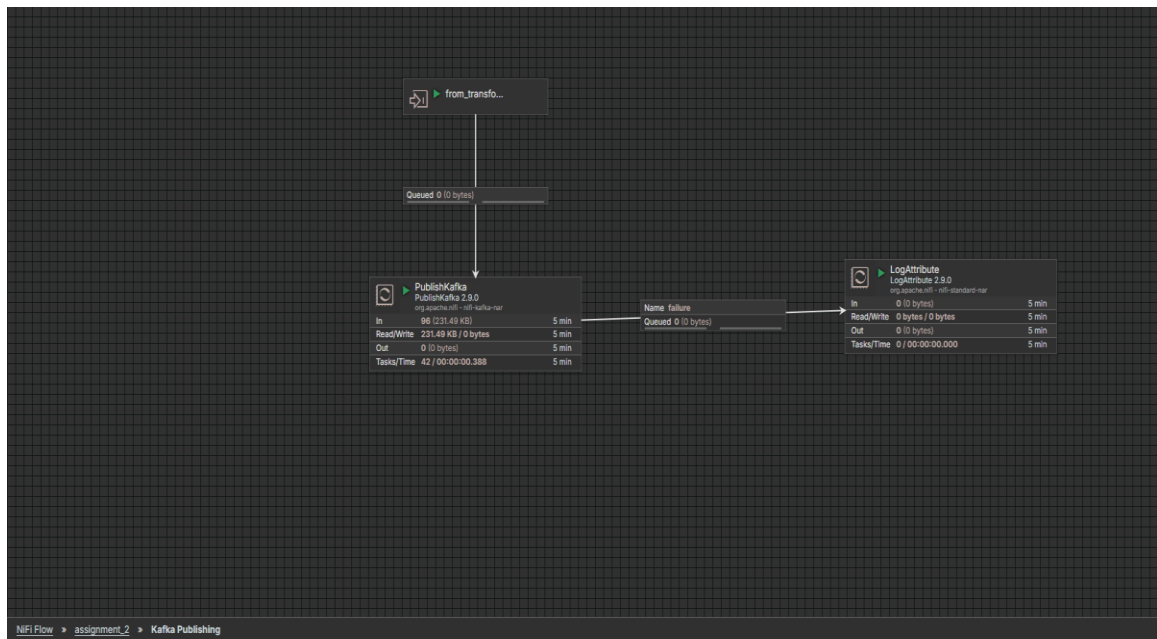
- **CSVReader**: Reads CSV files based on the header row.
- **JsonRecordSetWriter**: Writes the output in structured JSON format.
- **JsonTreeReader_Strict**: Reads the resulting JSON for strict schema validation.
- **Kafka3ConnectionService**: Shared Kafka connection.

4. Kafka Integration

a. Kafka Topic Creation

```
Select @kafka1~  
  
E:\DE_A1\Data Engineering Bootcamp\spark-sql-and-pyspark-using-python3>docker exec -it kafka1 bash  
[appuser@kafka1 ~]$ kafka-topics --create --topic smart_metering_readings --bootstrap-server localhost:19092 --partitions 5 --replication-factor 1  
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.  
Created topic smart_metering_readings.  
[appuser@kafka1 ~]$  
Display all 658 possibilities? (y or n)  
[appuser@kafka1 ~]$ kafka-topics --list --bootstrap-server localhost:19092  
ahmed  
smart_metering_readings  
[appuser@kafka1 ~]$
```

b. Publishing messages from NiFi



c. Consuming messages from Kafka

```
C:\Windows\System32\cmd.exe
IDE AI[Data Engineering Bootcamp]\spark-sql-and-gyrfark-using-python\docker exec -it kafka1 kafka-console-consumer --bootstrap-server localhost:19092 --topic smart_metering_readings --from-beginning
{"meter_id":"MTR1002","building_id":"BLD2005","power_kw":11.27,"voltage":null,"current":9.58,"status":"FAULT","timestamp":"15/05/2026 22:36"}
{"meter_id":"MTR1009","building_id":"BLD2000","power_kw":6.08,"voltage":110,"current":4.29,"status":"INACTIVE","timestamp":"2026-05-15 23:28:28"}
{"meter_id":"MTR1009","building_id":"BLD2000","power_kw":6.08,"voltage":110,"current":4.29,"status":"INACTIVE","timestamp":"2026-05-15 23:28:28"}
{"meter_id":"MTR1001","building_id":"BLD2007","power_kw":4.6,"voltage":110,"current":5.55,"status":"INACTIVE","timestamp":"05-15-2026 22:40"}
{"meter_id":"MTR1004","building_id":"BLD2004","power_kw":2.2,"voltage":230,"current":43.43,"status":"FAULT","timestamp":"2026-05-15 22:19:05"}
{"meter_id":"MTR1004","building_id":"BLD2004","power_kw":2.2,"voltage":230,"current":43.43,"status":"FAULT","timestamp":"2026-05-15 22:19:05"}
{"meter_id":"MTR1008","building_id":"BLD2007","power_kw":12.1,"voltage":230,"current":1.14,"status":"INACTIVE","timestamp":"2026-05-15 23:15:57"}
{"meter_id":"MTR1006","building_id":"BLD2002","power_kw":5.73,"voltage":220,"current":8.7,"status":"INACTIVE","timestamp":"INVALID DATE"}
{"meter_id":"MTR1001","building_id":"BLD2004","power_kw":17.98,"voltage":230,"current":42.91,"status":"ACTIVE","timestamp":"05-15-2026 22:43"}
{"meter_id":"MTR1005","building_id":"BLD2100","power_kw":1.11,"voltage":110,"current":19.46,"status":"FAULT","timestamp":"2026-05-15 21:11:48"}
{"meter_id":"MTR1002","building_id":"BLD2002","power_kw":13.33,"voltage":220,"current":10.23,"status":"ACTIVE","timestamp":"05-15-2026 21:30"}
{"meter_id":"MTR1002","building_id":"BLD2100","power_kw":0.72,"voltage":110,"current":12.45,"status":"FAULT","timestamp":"2026-05-15 22:31:42"}
{"meter_id":"MTR1007","building_id":"BLD2009","power_kw":4.27,"voltage":230,"current":24.53,"status":"ACTIVE","timestamp":"15/05/2026 22:54"}
{"meter_id":"MTR1006","building_id":"BLD2000","power_kw":13.09,"voltage":110,"current":23.2,"status":"INACTIVE","timestamp":"2026-05-15 22:54:52"}
{"meter_id":"MTR1006","building_id":"BLD2000","power_kw":13.09,"voltage":110,"current":23.2,"status":"INACTIVE","timestamp":"2026-05-15 22:54:52"}
{"meter_id":"MTR1001","building_id":"BLD2100","power_kw":1.79,"voltage":110,"current":40.22,"status":"ACTIVE","timestamp":"2026-05-15 22:54:47"}
{"meter_id":"MTR1009","building_id":"BLD2000","power_kw":18.7,"voltage":null,"current":27.19,"status":"ACTIVE","timestamp":"15/05/2026 20:51"}
{"meter_id":"MTR1003","building_id":"BLD2004","power_kw":9.3,"voltage":null,"current":24.32,"status":"ACTIVE","timestamp":"2026-05-15 21:55:30"}
{"meter_id":"MTR1006","building_id":"BLD2008","power_kw":5.03,"voltage":220,"current":48.63,"status":"FAULT","timestamp":"15/05/2026 22:51"}
{"meter_id":"MTR1001","building_id":"BLD2005","power_kw":16.01,"voltage":null,"current":12.29,"status":"INACTIVE","timestamp":"2026-05-15 21:02:49"}
{"meter_id":"MTR1008","building_id":"BLD2005","power_kw":1.4,"voltage":230,"current":16.35,"status":"ACTIVE","timestamp":"2026-05-15 22:45:30"}
{"meter_id":"MTR1007","building_id":null,"power_kw":17.05,"voltage":null,"current":34.09,"status":"INACTIVE","timestamp":"15/05/2026 20:54"}
{"meter_id":"MTR1001","building_id":"BLD2006","power_kw":5.54,"voltage":230,"current":44.87,"status":"FAULT","timestamp":"15/05/2026 21:58"}
{"meter_id":"MTR1010","building_id":"BLD2003","power_kw":10.66,"voltage":null,"current":40.86,"status":"FAULT","timestamp":"2026-05-15 22:42:10"}
{"meter_id":"MTR1008","building_id":"BLD2008","power_kw":17.82,"voltage":null,"current":40.12,"status":"ACTIVE","timestamp":"15/05/2026 21:54"}
{"meter_id":"MTR1002","building_id":"BLD2006","power_kw":9.42,"voltage":null,"current":42.93,"status":"FAULT","timestamp":"05-15-2026 21:29"}
{"meter_id":"MTR1010","building_id":"BLD2009","power_kw":15.22,"voltage":230,"current":14.66,"status":"FAULT","timestamp":"INVALID DATE"}
{"meter_id":"MTR1007","building_id":"BLD2002","power_kw":12.03,"voltage":9999,"current":16.17,"status":"FAULT","timestamp":"15/05/2026 22:56"}
{"meter_id":"MTR1003","building_id":"BLD2008","power_kw":18.48,"voltage":null,"current":13.46,"status":"FAULT","timestamp":"INVALID DATE"}
{"meter_id":"MTR1003","building_id":"BLD2008","power_kw":18.48,"voltage":null,"current":13.46,"status":"FAULT","timestamp":"INVALID DATE"}
{"meter_id":"MTR1001","building_id":"BLD2001","power_kw":5.66,"voltage":null,"current":31.67,"status":"INACTIVE","timestamp":"15/05/2026 21:25"}
{"meter_id":"MTR1002","building_id":"BLD2002","power_kw":5.32,"voltage":null,"current":32.98,"status":"ACTIVE","timestamp":"INVALID DATE"}
{"meter_id":"MTR1010","building_id":"BLD2008","power_kw":19.51,"voltage":9999,"current":15.03,"status":"ACTIVE","timestamp":"INVALID DATE"}
{"meter_id":"MTR1009","building_id":"BLD2002","power_kw":13.0,"voltage":110,"current":9.19,"status":"ACTIVE","timestamp":"2026-05-15 22:47:05"}
{"meter_id":"MTR1000","building_id":"BLD2005","power_kw":13.97,"voltage":220,"current":39.93,"status":"FAULT","timestamp":"2026-05-15 21:45:23"}
{"meter_id":"MTR1002","building_id":"BLD2004","power_kw":5.64,"voltage":230,"current":27.68,"status":"ACTIVE","timestamp":"2026-05-15 22:27:06"}
{"meter_id":"MTR1002","building_id":"BLD2008","power_kw":15.5,"voltage":110,"current":13.93,"status":"FAULT","timestamp":"2026-05-15 23:27:21"}
{"meter_id":"MTR1003","building_id":"BLD2002","power_kw":8.93,"voltage":110,"current":12.61,"status":"ACTIVE","timestamp":"INVALID DATE"}
{"meter_id":"MTR1006","building_id":"BLD2009","power_kw":9.73,"voltage":230,"current":18.63,"status":"FAULT","timestamp":"05-15-2026 23:04"}
{"meter_id":"MTR1003","building_id":null,"power_kw":17.5,"voltage":110,"current":3.75,"status":"INACTIVE","timestamp":"15/05/2026 22:47"}
{"meter_id":"MTR1006","building_id":"BLD2001","power_kw":12.51,"voltage":220,"current":49.18,"status":"INACTIVE","timestamp":"INVALID DATE"}
{"meter_id":"MTR1009","building_id":"BLD2006","power_kw":5.96,"voltage":110,"current":1.15,"status":"INACTIVE","timestamp":"05-15-2026 22:24"}
{"meter_id":"MTR1006","building_id":"BLD2009","power_kw":14.24,"voltage":110,"current":17.65,"status":"ACTIVE","timestamp":"2026-05-15 21:16:34"}
{"meter_id":"MTR1004","building_id":"BLD2002","power_kw":13.55,"voltage":230,"current":26.35,"status":"ACTIVE","timestamp":"INVALID DATE"}
{"meter_id":"MTR1004","building_id":"BLD2002","power_kw":13.55,"voltage":230,"current":26.35,"status":"ACTIVE","timestamp":"INVALID DATE"}
{"meter_id":"MTR1010","building_id":"BLD2003","power_kw":4.43,"voltage":null,"current":45.37,"status":"FAULT","timestamp":"2026-05-15 22:59:29"}
{"meter_id":"MTR1003","building_id":"BLD2004","power_kw":18.82,"voltage":220,"current":37.82,"status":"INACTIVE","timestamp":"05-15-2026 22:04"}
{"meter_id":"MTR1001","building_id":"BLD2007","power_kw":11.39,"voltage":230,"current":9.94,"status":"INACTIVE","timestamp":"15/05/2026 22:42"}
```

d. Kafka UI

UI for Apache Kafka83b5a60 v0.7.2

Dashboard

local

Brokers

Topics

Consumers

ACL

Topics

Search by Topic Name

Show Internal Topics

Generate random topicsCopy all random topicsExport all random topics to CSV

Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size
ahmed	3	0	1	0	0 Bytes
smart_metering_readings	5	0	1	0	0 Bytes

5. Hadoop Storage

I designed a storage path that consumes data from Kafka and stores it in HDFS with dynamic date-based partitioning.

UpdateAttribute allows me to dynamically build the HDFS path using **NiFi Expression Language** before writing to HDFS, then **PutHDFS** can use them automatically in its properties configurations.

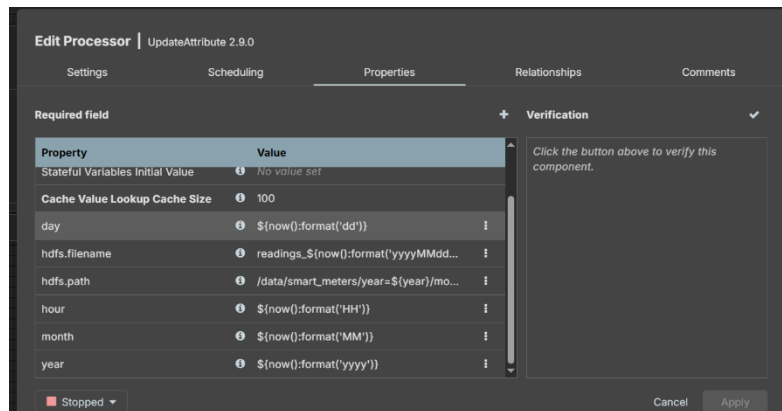


Figure 3: UpdateAttribute Properties Configurations

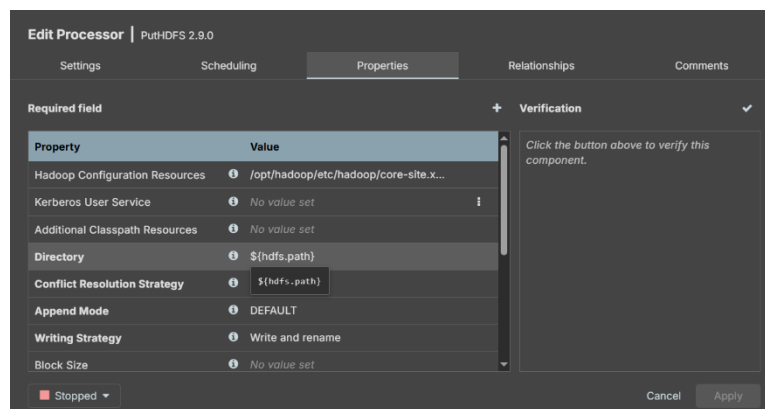


Figure 4: PutHDFS Properties Configurations

```
C:\Windows\System32\cmd.exe
E:\DE_A1\Data Engineering Bootcamp\spark-sql-and-pyspark-using-python3-docker compose exec itvdelab hdfs dfs -ls -R /data

drwxr-xr-x  - itversity supergroup 0 2026-05-15 22:22 /data/smart_meters
drwxr-xr-x  - itversity supergroup 0 2026-05-15 22:22 /data/smart_meters/year=
drwxr-xr-x  - itversity supergroup 0 2026-05-15 22:22 /data/smart_meters/year=/month=
drwxr-xr-x  - itversity supergroup 0 2026-05-15 22:22 /data/smart_meters/year=/month=/day=
drwxr-xr-x  - itversity supergroup 0 2026-05-15 22:22 /data/smart_meters/year=/month=/day=/hour=
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/12b084e5-da15-41fc-a157-6dbf20b7da6f.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/273809938837731.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/273805941156499.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/2738159486317611.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/27382595542250.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/273830958635782.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/273840964581083.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/273845969253899.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/27385594908752.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/273860978134680.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/273865981057670.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/27387598931128.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/552d807c-4b33-42c4-b00e-63fc6e721cb.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/949eba2c-2c9f-402c-a726-1911f434d6cd.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/990d67e2-86ce-4471-90ef-25468149a8c6.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:23 /data/smart_meters/year=/month=/day=/hour=/a1410a1e-c33e-4452-a114-735ce14d6601.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/d70c49ab-910c-4096-8046-8b0f5e5479f8.124
-rw-r--r-- 1 itversity supergroup 0 2026-05-15 22:24 /data/smart_meters/year=/month=/day=/hour=/f2c0c8af-e150-4f74-a982-b56a117ee17.124
-rw-r--r-- 1 itversity supergroup 2124 2026-05-15 22:32 /data/smart_meters/year=/month=/day=/hour=/01180d58-a860-4ffa-9a00-b10d725a6bb1
-rw-r--r-- 1 itversity supergroup 2604 2026-05-15 22:35 /data/smart_meters/year=/month=/day=/hour=/0181f6c5-4e65-481f-a8ca-c51b039356e8
-rw-r--r-- 1 itversity supergroup 2486 2026-05-15 22:32 /data/smart_meters/year=/month=/day=/hour=/086324fc-c2da-4b54-blee-9eb3c3daced8f
-rw-r--r-- 1 itversity supergroup 2787 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/0da9a3a8-c320-4c6e-a843-06fc73cd4d6
-rw-r--r-- 1 itversity supergroup 2133 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/12b084e5-da15-41fc-a157-6dbf20b7da6f
-rw-r--r-- 1 itversity supergroup 2835 2026-05-15 22:35 /data/smart_meters/year=/month=/day=/hour=/13572ede-270e-4d0a-adf6-3f933fee1a5d
-rw-r--r-- 1 itversity supergroup 1861 2026-05-15 22:35 /data/smart_meters/year=/month=/day=/hour=/1341e899-0c30-4b07-ab071-f533d1ceb528
-rw-r--r-- 1 itversity supergroup 2681 2026-05-15 22:31 /data/smart_meters/year=/month=/day=/hour=/259ed334-401a-4625-be68-20bb8b4302c4
-rw-r--r-- 1 itversity supergroup 5524 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/27382595542250
-rw-r--r-- 1 itversity supergroup 5543 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/273830958635782
-rw-r--r-- 1 itversity supergroup 4784 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/273840964581083
-rw-r--r-- 1 itversity supergroup 5220 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/273845969253899
-rw-r--r-- 1 itversity supergroup 4684 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/274121421852651
-rw-r--r-- 1 itversity supergroup 204687 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/274126433422799
-rw-r--r-- 1 itversity supergroup 5125 2026-05-15 22:28 /data/smart_meters/year=/month=/day=/hour=/274134521246997
-rw-r--r-- 1 itversity supergroup 37761 2026-05-15 22:29 /data/smart_meters/year=/month=/day=/hour=/274186093365587
-rw-r--r-- 1 itversity supergroup 5385 2026-05-15 22:29 /data/smart_meters/year=/month=/day=/hour=/274196099576272
-rw-r--r-- 1 itversity supergroup 4486 2026-05-15 22:29 /data/smart_meters/year=/month=/day=/hour=/274201102834457
-rw-r--r-- 1 itversity supergroup 5823 2026-05-15 22:29 /data/smart_meters/year=/month=/day=/hour=/274206100939633
-rw-r--r-- 1 itversity supergroup 4947 2026-05-15 22:29 /data/smart_meters/year=/month=/day=/hour=/274210114904716
-rw-r--r-- 1 itversity supergroup 4935 2026-05-15 22:30 /data/smart_meters/year=/month=/day=/hour=/274220121252488
-rw-r--r-- 1 itversity supergroup 5530 2026-05-15 22:30 /data/smart_meters/year=/month=/day=/hour=/274231124230149
-rw-r--r-- 1 itversity supergroup 5829 2026-05-15 22:30 /data/smart_meters/year=/month=/day=/hour=/274241131016059
-rw-r--r-- 1 itversity supergroup 2503 2026-05-15 22:30 /data/smart_meters/year=/month=/day=/hour=/274246133515366
```

Figure 5: Hadoop stores file by data partitions

NOTE: I used MergeContent to solve the problem of small files by combining them into Large File.

Here is the HDFS storage Nifie work flow :

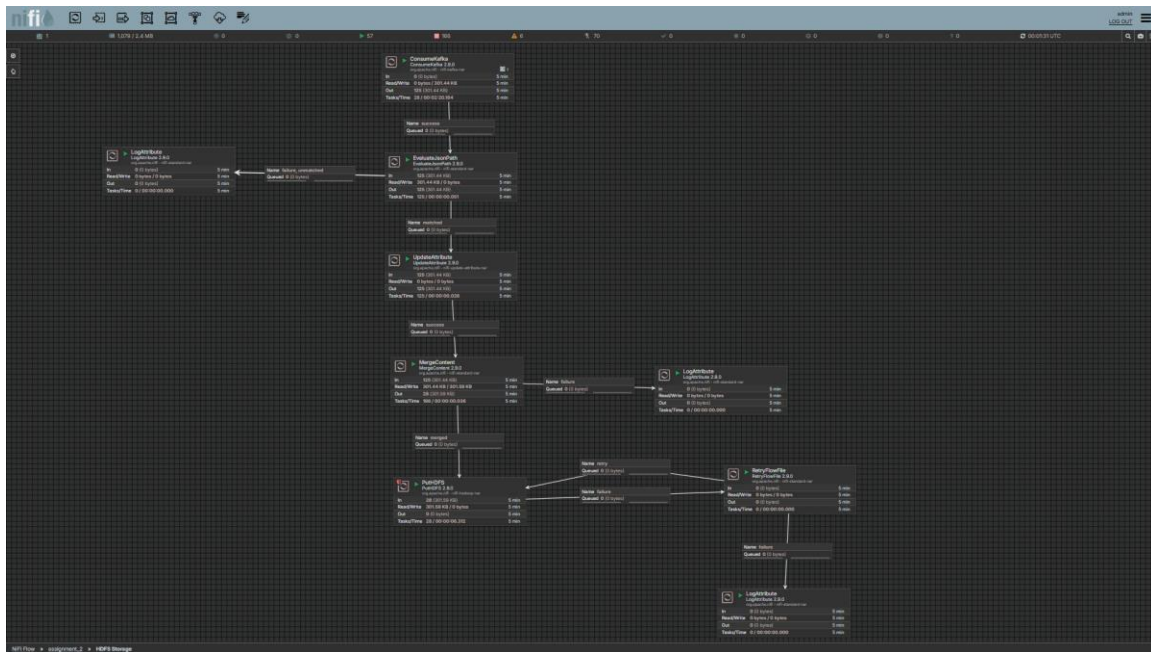


Figure 6: Hadoop stores file by data partitions

6- Monitoring and Reliability

I tried to follow best practices for monitoring and error handling, here are the types of errors I handled:

Error Type	Handling Mechanism
Temporary failures (network, file locked)	RetryFlowFile — 3 attempts, then log
Data quality issues (NaN, INF values)	Routed to to_monitoring → HDFS
Corrupt data (unparseable chunks)	Routed to to_errors → HDFS

So, No data is discarded. Clean records go to Kafka and HDFS, flagged records with anomalous values are stored under /data/monitoring/ for review, and corrupt chunks are stored under /data/errors/ for later analysis.

That way is better than dragging LogAttribute processors everywhere which causes severe disk I/O bottlenecks and visually clutters the pipeline.

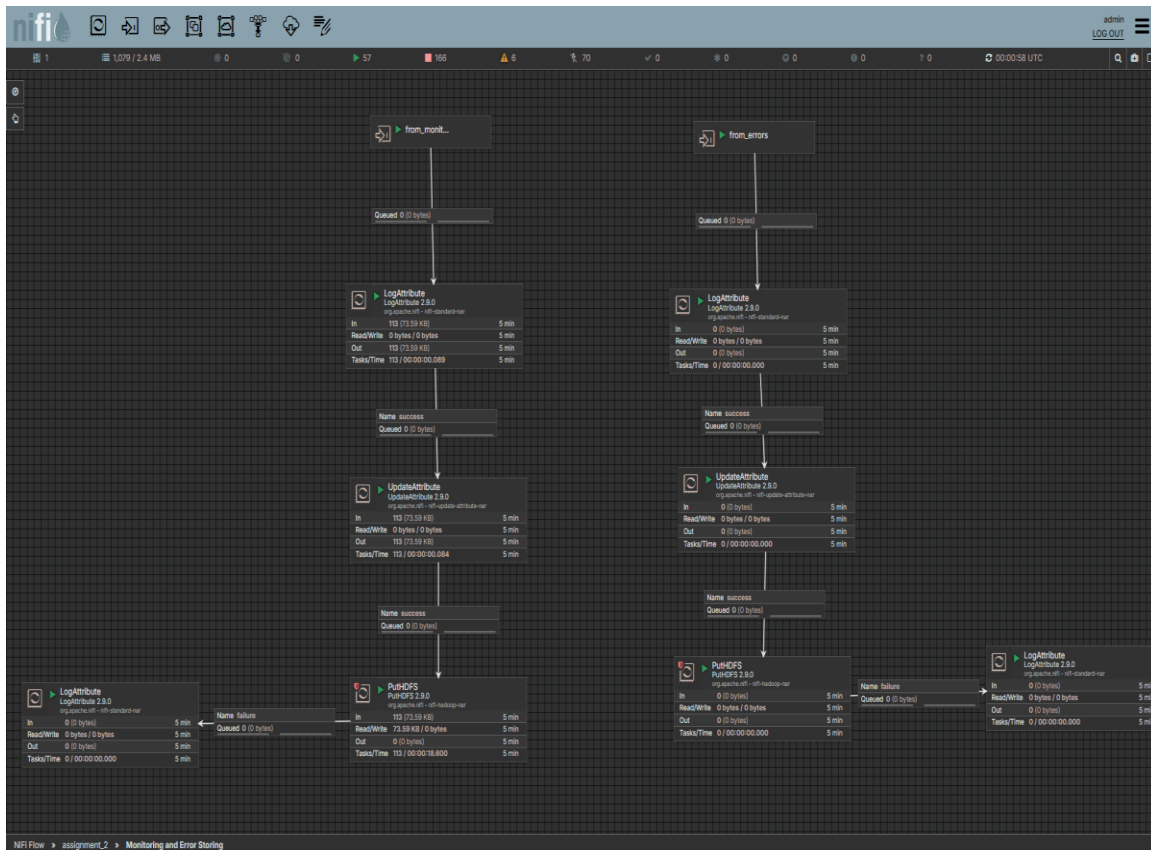


Figure 7: NIFI workflow for monitoring and error storage

The flowFile of the Entire Project in Processor Groups:

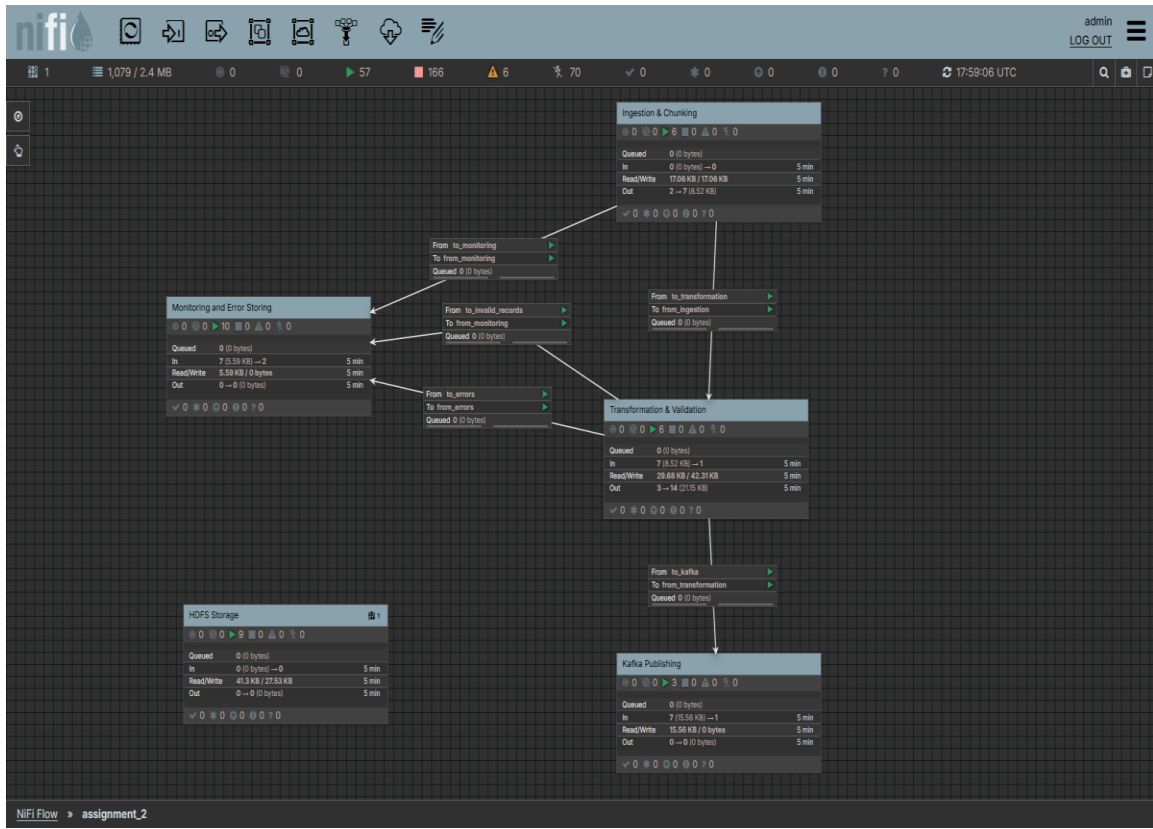
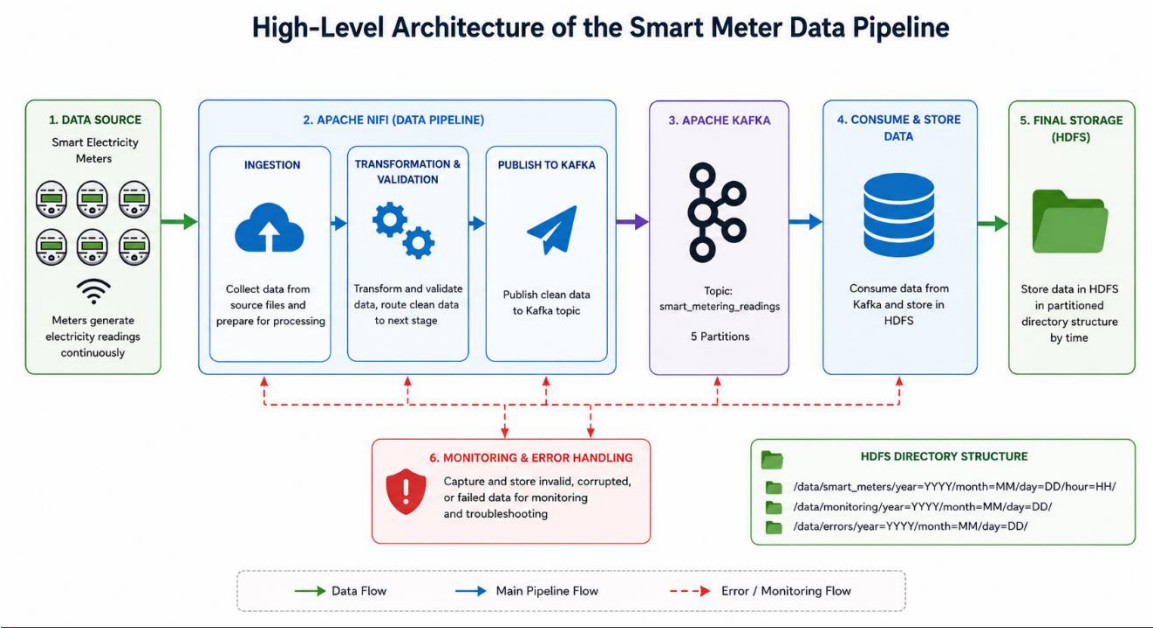


Figure 8: General Workflow of the project

Complete Architecture of The Pipeline:



NOTE: AI was only used to quickly generate the visual representation of the architecture diagram. The pipeline architecture itself, including its design and implementation details, was entirely created based on my own project work.